Softwarearchitektur

- CrossNote

Datum:	14.01.2026
Version:	1.0
Prüfer:	Nicolas Kauer
Freigegeben von:	Jakob Scheuermeyer (Qualitätsmanager)

Changelog:

1.0: Deckblatt, grobe Ausarbeitung

Gerhard Uebe, Nicolas Kauer, Jakob Scheuermeyer, Kai Seitz

1. Zweck und Scope

CrossNote ist eine plattformübergreifende Notiz-Anwendung für **Desktop (Windows/Linux)** und **Android**. Kernprinzip ist **Offline-First**: Alle Funktionen (Notizen, Tags/Ordner, Suche, Revisionen, Papierkorb) laufen ohne Internet. Optional werden Änderungen später über einen Synchronisationsdienst abgeglichen, inklusive Konfliktbehandlung.

Die Architektur ist so ausgelegt, dass:

- UI plattformspezifisch bleibt, Logik jedoch plattformneutral,
- Daten lokal zuverlässig gespeichert werden,
- Suche schnell bleibt,
- Sync modular nachrüstbar/erweiterbar ist,
- Java und Kotlin gemeinsam nutzbar sind (JVM-Interop)

.2. Architekturtreiber und Qualitätsziele

2.1 Funktionale Treiber

- Notiz erstellen/bearbeiten/löschen (Papierkorb), wiederherstellen
- Tags + Notizbücher/Ordner verwalten
- Revisionen / Verlauf / Restore
- Suche/Filter (lokal)
- Export/Import und Backups
- Optional: Sync (Push/Pull), Offline-Queue, Konfliktauflösung

2.2 Nicht-funktionale Qualitätsziele

- **Offline-Fähigkeit:** volle Nutzbarkeit ohne Netzwerk.
- **Zuverlässigkeit:** keine Datenverluste bei Crash/Abbruch (Transaktionen, atomare Updates).
- **Performance:** schnelle Suche/Filter (lokaler Index/FTS).
- **Sicherheit:** verschlüsselte Backups und verschlüsselte Sync-Kommunikation.
- **Erweiterbarkeit:** neue Notiztypen/Anhänge/Features ohne große Umbauten.
- **Portabilität:** gleiche Core-Logik auf Desktop und Android.

3. Technologiestack (JavaFX + Kotlin)

Beschreibung

In diesem Kapitel wird dargelegt, auf welcher Plattform welche Programmieroberflächen für welche Zwecke verwendet werden.

Desktop (Windows/Linux)

- **JavaFX** für UI
- JVM: Java (UI) + Kotlin (Core optional)
- Lokale Datenbank: **SQLite**

Android

- UI: Android UI (z. B. Jetpack Compose oder XML Views; hier nur als “Android UI” geführt)
- App-Layer/Core: **Kotlin**
- Lokale Datenbank: **SQLite** (z. B. über Room oder direkte SQLite-Adapter)

Gemeinsamer Core

- Domain + Use-Case-Logik in einem **shared Core-Modul** (Kotlin empfohlen, weil sehr gut auf Android & JVM passt; Java-Interop für Desktop ist problemlos)

4. Architekturstil

CrossNote verwendet eine **Schichtenarchitektur** mit klarer Abhängigkeitsrichtung:

UI (Desktop/Android) → Application (Use Cases) → Domain (Core) → Infrastructure (DB/Files/Sync/Search)

Abhängigkeiten laufen **nur nach unten**.
Infrastruktur wird über **Interfaces (Ports)** angebunden, sodass:

- DB/Suche/Sync austauschbar sind,
- Unit-Tests ohne echte DB/Netzwerk möglich sind.

5. Sichtenmodell (4+1)

Beschreibung

In diesem Kapitel werden die Kern-Szenarien dargelegt, die durch ihre Relevanz für die Software ausschlaggebend für dieArchitektur der Software sind

5.1 Szenario-Sicht (+1)

Kern-Szenarien, die die Architektur “treiben”:

1. Notiz erstellen/bearbeiten (Autosave + Revision)
2. Papierkorb (Soft Delete) + Restore
3. Suche/Filter (lokal, performant)
4. Sync (Outbox/Pull) + Konfliktauflösung
5. Export/Import + Backup/Restore

6. Logische Sicht (Domänenmodell)

Beschreibung

In diesem Kapitel wird die Software in Objekte und Regeln unterteilt. Diese charakterisieren die Domäne, in unserem Fall das ganze Projekt.

6.1 Zentrale Domänenobjekte

- **Note**
  - `id`, `title`, `content`, `type`, `createdAt`, `updatedAt`
  - `notebookId`, `tags[]`, `isPinned`, `isFavorite`
  - `trashedAt` (null = aktiv, gesetzt = im Papierkorb)
  - `syncState` (PENDING / SYNCED / CONFLICT / ERROR)

- **Notebook**
  - `id`, `name`, `createdAt`

- **Tag**
  - `id`, `name`

- **Attachment**
  - `id`, `noteId`, `kind`, `path`, `hash`, `metadata`

- **Revision**
  - `id`, `noteId`, `timestamp`, `snapshot` (oder diff)

- **OutboxItem** (Offline-Queue)
  - `id`, `type`, `payload`, `createdAt`, `retryCount`

- **Conflict**
  - `noteId`, `localRevision`, `remoteRevision`, `strategy`

6.2 Domänenregeln

- Änderungen erzeugen Revisionen nach Policy.
- Papierkorb ist Soft Delete (physisches Löschen optional als “Purge”).
- Offline-First: zuerst lokal committen, dann Outbox befüllen.
- Konflikte sind “first-class” und blockieren Sync für betroffene Notizen, bis entschieden.

7. Implementierungssicht (Komponenten, Ports, Adapter)

Beschreibung

Dieses Kapitel unterteilt das Projekt in vier Schichten. Diese beziehen sich auf die Benutzeroberfläche, Verarbeitungsschicht, Domäne und die restliche Infrastruktur.

7.1 Schicht 1 – UI / Presentation

Desktop UI (JavaFX, MVVM)

- Views: `NoteListView`, `NoteEditorView`, `TagNotebookView`, `SettingsView`, `ConflictDialog`
- ViewModels: `NoteListVM`, `EditorVM`, `SyncStatusVM`
  UI kennt nur Application-Services (Use Cases).

Android UI (MVVM)

- Screens: NoteList, Editor, Tags/Notebooks, Settings, Conflicts
- ViewModels: analog zu Desktop
  Android UI kennt nur Application-Services (Use Cases).

Wichtig: UI enthält keine DB-/Sync-Logik.

7.2 Schicht 2 – Application (Use-Case-Schicht)

Orchestriert Use Cases, Validierung, Transaktionsgrenzen, und ruft Ports auf.

Services (Beispiele):

- `NoteAppService`
  - `createNote()`, `updateNote()`, `moveToTrash()`, `restoreFromTrash()`, `listNotes(filter)`

- `SearchAppService`
  - `search(query, filters)`

- `RevisionAppService`
  - `listRevisions(noteId)`, `restoreRevision(noteId, revisionId)`

- `ExportImportAppService`
  - `export(format, target)`, `import(source)`

- `SyncAppService`
  - `syncNow()`, `getStatus()`, `resolveConflict(noteId, strategy)`

7.3 Schicht 3 – Domain (Core)

Enthält Entities, Value Objects, Domain-Services/Policies:

- `RevisionPolicy` (wann/was speichern)
- `ConflictResolver` (KEEP_LOCAL / KEEP_REMOTE / MERGE)
- `SyncPolicy` (Retry/Backoff)
- `NoteValidator`

7.4 Schicht 4 – Infrastructure (Adapter/Implementierungen)

Persistence (SQLite)

- `SqliteNoteRepository`
- `SqliteTagRepository`
- `SqliteNotebookRepository`
- `SqliteRevisionRepository`
- `SqliteOutboxRepository`

Search/Index

- `SqliteFtsSearchIndex` oder eigener `LocalSearchIndex`
- Update bei Save/Delete

File Storage

- `LocalFileStorage` (Attachments, Export/Import, Backups)

Sync

- `HttpSyncClient` (REST/JSON über HTTPS)
- `SyncEngine` (Outbox push, pull, merge, conflicts erstellen)

8. Ports (Interfaces) – Plattformneutraler Kern

Beschreibung

In diesem Kapitel wird die Position der Interfaces deklariert, die für die Kernkommunikation der Software zuständig sind.

Diese Interfaces liegen im shared Core (Domain/Application), nicht in Infrastruktur:

```
interface NoteRepository {

  Note save(Note note);

  Optional<Note> findById(NoteId id);

  List<Note> list(NoteFilter filter);

  void softDelete(NoteId id, Instant trashedAt);

  void purge(NoteId id);

}


interface RevisionRepository {

  void add(Revision revision);

  List<Revision> listByNote(NoteId id);

  Optional<Revision> find(RevisionId id);

}


interface SearchIndex {

  void index(Note note);

  void remove(NoteId id);

  List<NoteId> search(String query, SearchFilter filter);

}
```

```
interface OutboxRepository {
  void enqueue(OutboxItem item);
  List<OutboxItem> nextBatch(int limit);
  void markDone(OutboxItemId id);
  void markFailed(OutboxItemId id, String reason);
}

interface SyncClient {
  RemoteSyncResult push(List<OutboxItem> items);
  RemoteChanges pull(Instant since);
}

interface FileStorage {
  Path storeAttachment(AttachmentData data);
  AttachmentData loadAttachment(Path path);
  void writeExport(Path target, byte[] data);
  byte[] readImport(Path source);
}
```

9. Ablaufsicht (wichtige dynamische Abläufe)

Hier werden die beiden Abläufe unserer App aufgeführt.

9.1 Notiz bearbeiten (Offline-First + Revision + Index + Outbox)

1. UI ruft `NoteAppService.updateNote()` auf
2. AppService validiert + startet persistente Operation (Transaktionsblock)
3. Domain erzeugt Revision nach `RevisionPolicy`
4. `NoteRepository.save(note)`
5. `RevisionRepository.add(revision)`
6. `SearchIndex.index(note)`
7. `OutboxRepository.enqueue(UPDATE_EVENT)`
8. Commit / Erfolg an UI

Warum so? Damit die App bei Crash niemals “halb” gespeichert ist und Sync später exakt weiß, was zu übertragen ist.

9.2 Sync (Push/Pull + Konflikt)

1. `SyncAppService.syncNow()`
2. `OutboxRepository.nextBatch()` liefert lokale Änderungen
3. `SyncClient.push()` sendet
4. `SyncClient.pull()` lädt Remote-Änderungen
5. `SyncEngine` vergleicht:
    - kein Konflikt → lokal anwenden + indexen + Outbox aktualisieren
    - Konflikt → `syncState = CONFLICT`, `Conflict` speichern
6. UI zeigt Konflikt; `resolveConflict(strategy)` setzt Ergebnis durch (lokal/remote/merge)

10. Physische Sicht (Deployment)

Diese Sicht zeigt die physischen Geräte auf denen die App verwendet werden kann oder welche gebraucht werden.

10.1 Desktop Client

- JavaFX App (JVM)
- SQLite lokal
- Lokaler Suchindex
- FileStorage (Anhänge/Backups)
- Optional: HTTPS Sync

10.2 Android Client

- Android App
- SQLite lokal (z. B. Room)
- Lokaler Suchindex
- App-Storage (Anhänge/Backups)
- Optional: HTTPS Sync

10.3 Sync-Server

- REST API (z. B. Kotlin Ktor / Spring)
- Speicherung: Notes + Revisionen + Attachments
- HTTPS

11. Modulstruktur

Dieser Abschnitt zeigt die einzelnen Module und in welcher Sprache diese geschrieben werden.

Gradle Multi-Module:

- `:core-domain` (**Kotlin**)
  Entities, Value Objects, Domain Services, Ports (Interfaces)

- `:app-application` (**Kotlin**)
  Use Cases / Application Services / DTOs

- `:infra-sqlite` (**Kotlin/Java**)
  SQLite Implementierungen der Repositories

- `:infra-search` (**Kotlin/Java**)
  Suchindex/FTS

- `:infra-sync` (**Kotlin**)
  SyncEngine + HttpSyncClient

- `:infra-files` (**Kotlin/Java**)
  FileStorage + Export/Import + Backup

- `:ui-desktop-javafx` (**Java**)
  JavaFX Views + ViewModels

- `:ui-android` (**Kotlin**)
  Android UI + ViewModels

- optional `:sync-server` (**Kotlin**)
  Server API

Regel: UI-Module dürfen nur `:app-application` kennen, nicht `infra-*`.

12. Architekturentscheidungen

Beschreibung

In diesem Kapitel werden die Architekturentscheidungen nochmal komprimiert in einer kurzen Liste wiedergegeben.

1. **Schichtenarchitektur** (UI→App→Domain→Infra) für klare Verantwortlichkeiten und Testbarkeit.

2. **Shared Core** (Domain + Use Cases) für Desktop und Android, um doppelte Logik zu vermeiden.

3. **MVVM** auf beiden Plattformen, damit UI und Logik sauber getrennt sind.

4. **JSON/SQLite lokal** als robuste Offline-Persistenz.

5. **Outbox Pattern** für Offline-Sync: Änderungen werden lokal committen und später übertragen.

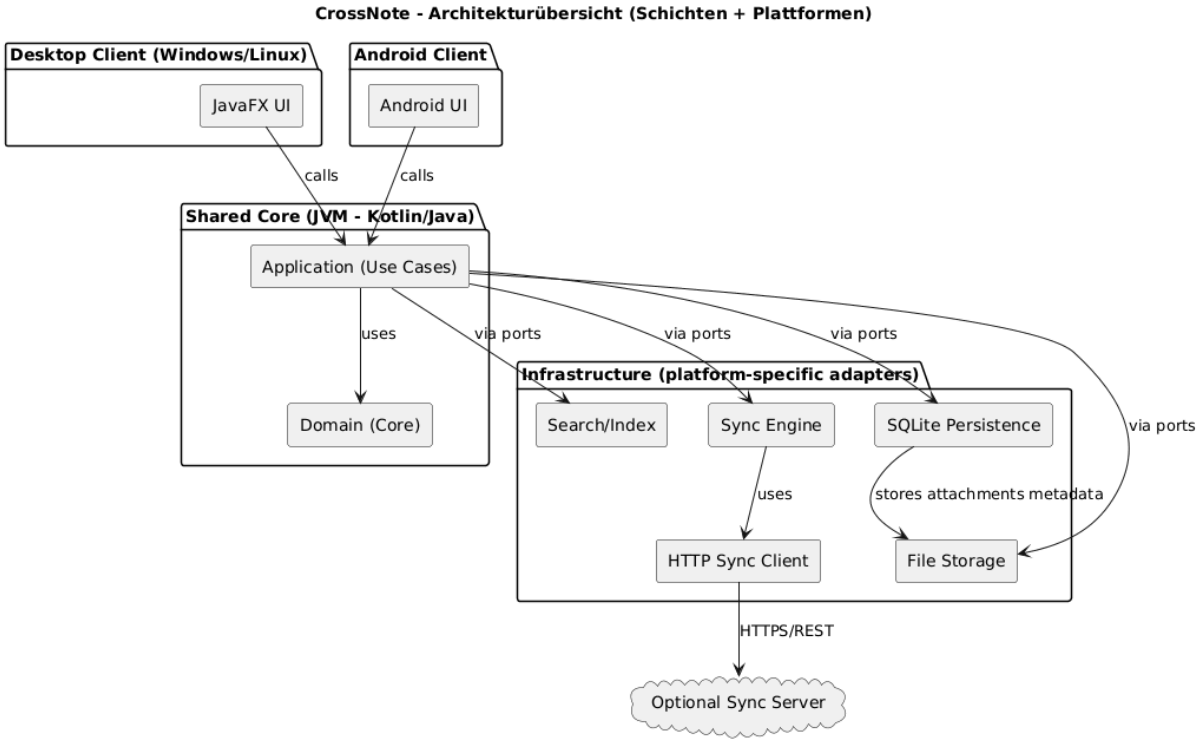
6. **Lokaler Suchindex** (FTS/Index) für schnelle Suche ohne Server.

7. **Konflikte als eigener Zustand** mit expliziter Nutzerentscheidung (Strategie).

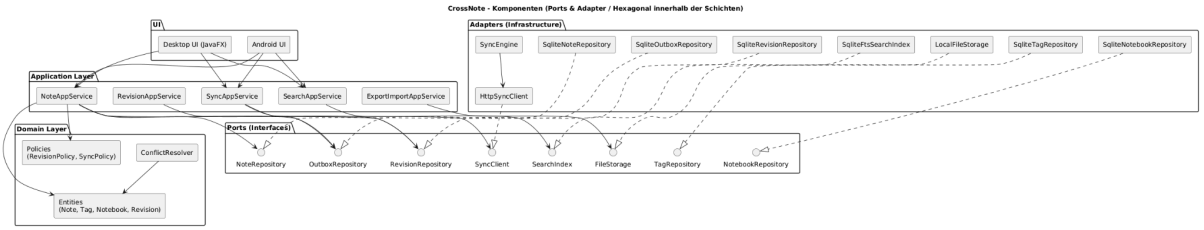
8. **Kotlin + Java Interop**: Kotlin im Core/Android, JavaFX UI in Java; beides läuft stabil auf der JVM.

13. Diagramme

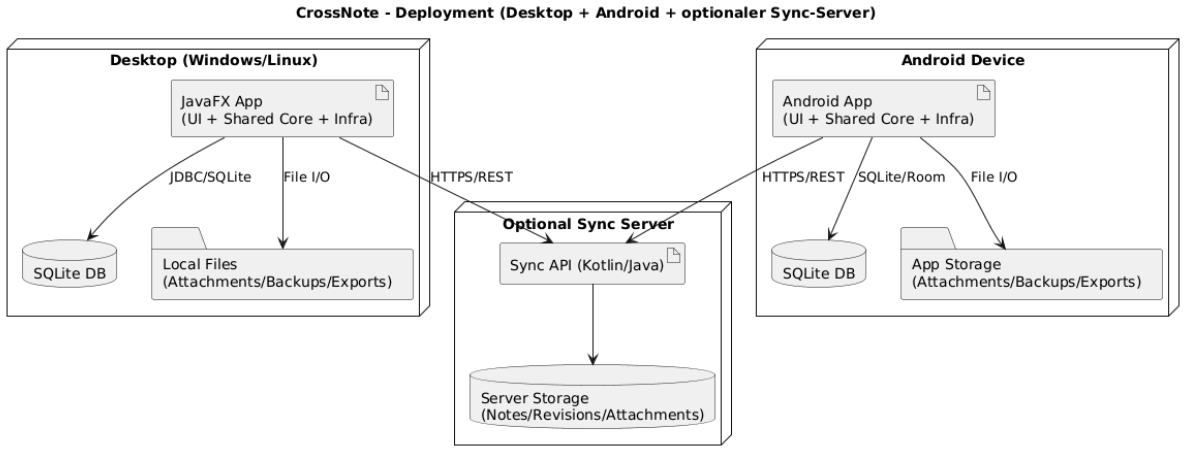
Beschreibung:
Dieses Diagramm beschreibt die Architekturübersicht der App Crossnote, besonders kann man an diesem Diagramm die Kommunikation der verschiedenen Geräte (Windows/Linux/Android) erkennen.



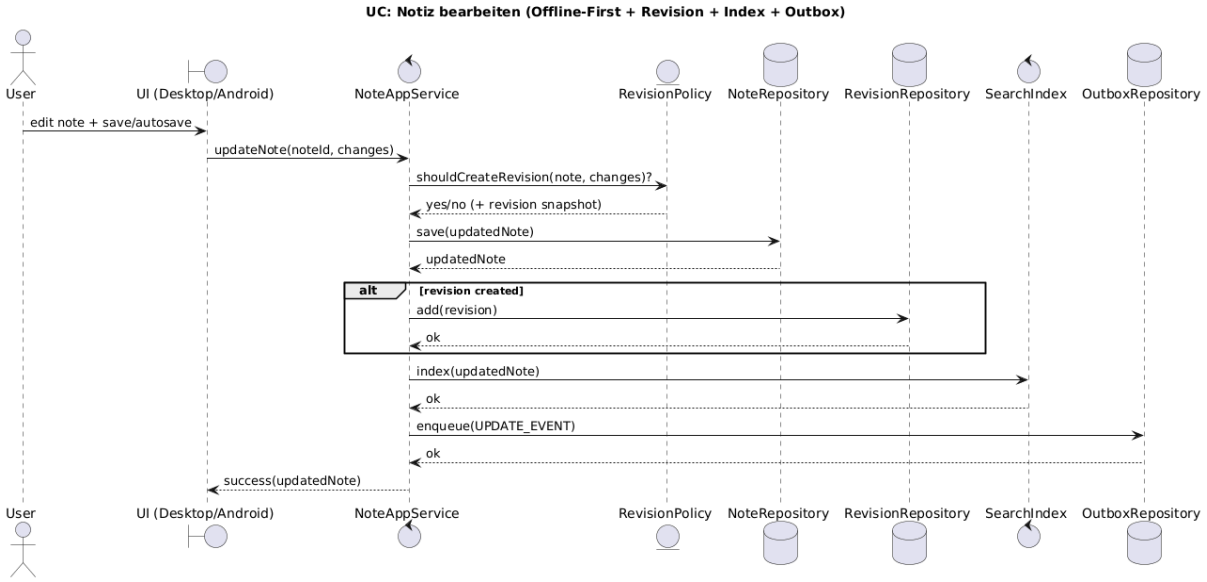
Beschreibung:
In diesem Diagramm werden die Komponenten näher beschrieben und dargestellt, sowie die Ports, welche die Komponenten benutzen.



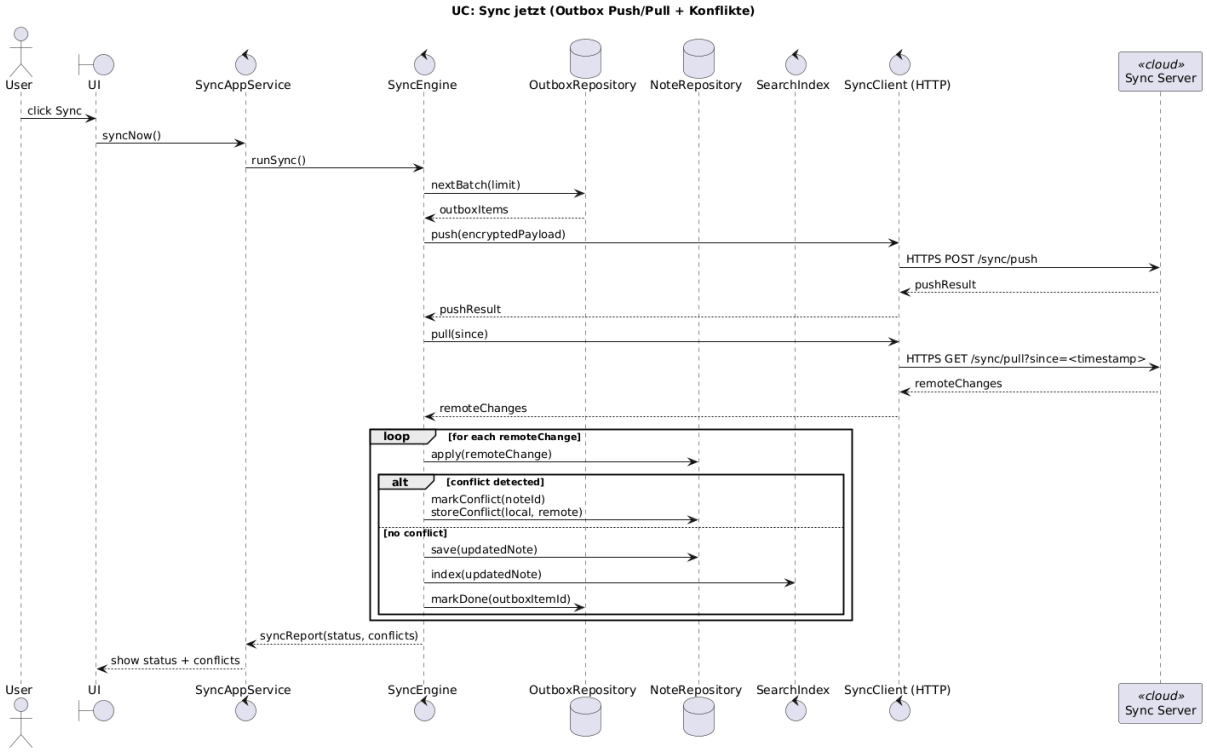
Beschreibung:
Dieses Diagramm stellt die Deployment Architektur der Crossnote App dar. Android und Windows/Linux verwenden eine sehr ähnliche Architektur, dies kann die Implementierung erleichtern.



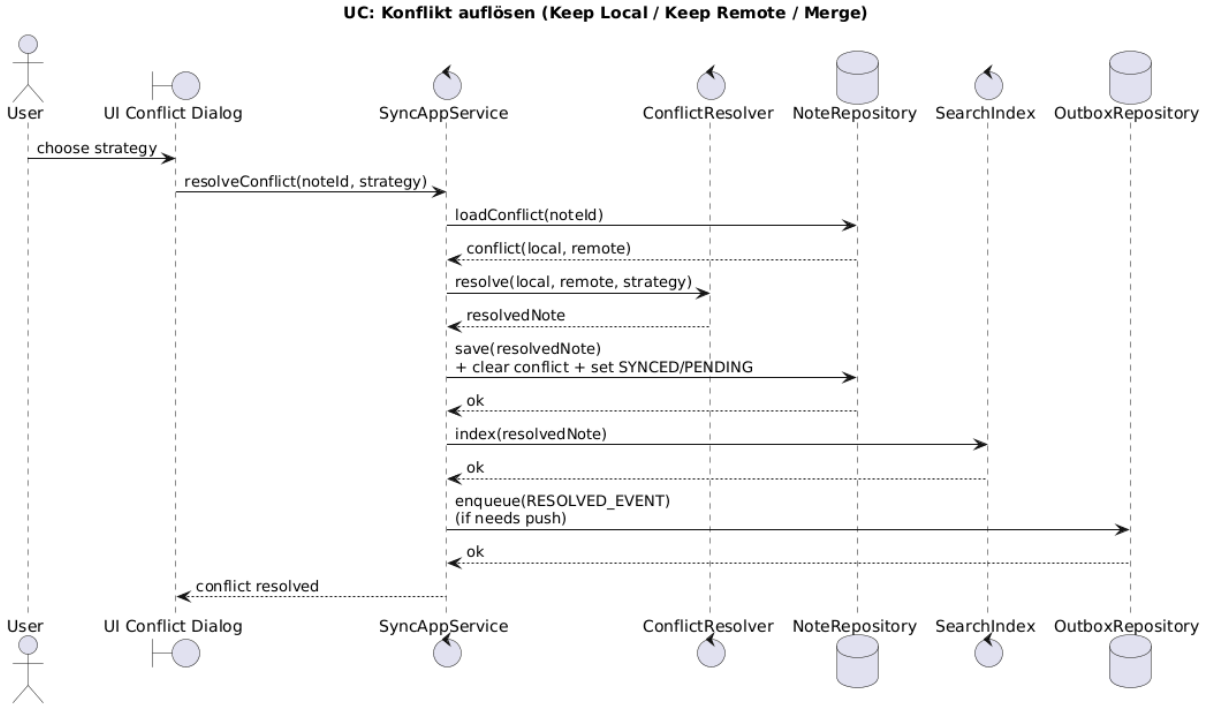
Beschreibung:
Dieses Diagramm stellt den Prozess dar, der ausgeführt wird, wenn eine Notiz bearbeitet wird und welche Schritte nacheinander ablaufen.



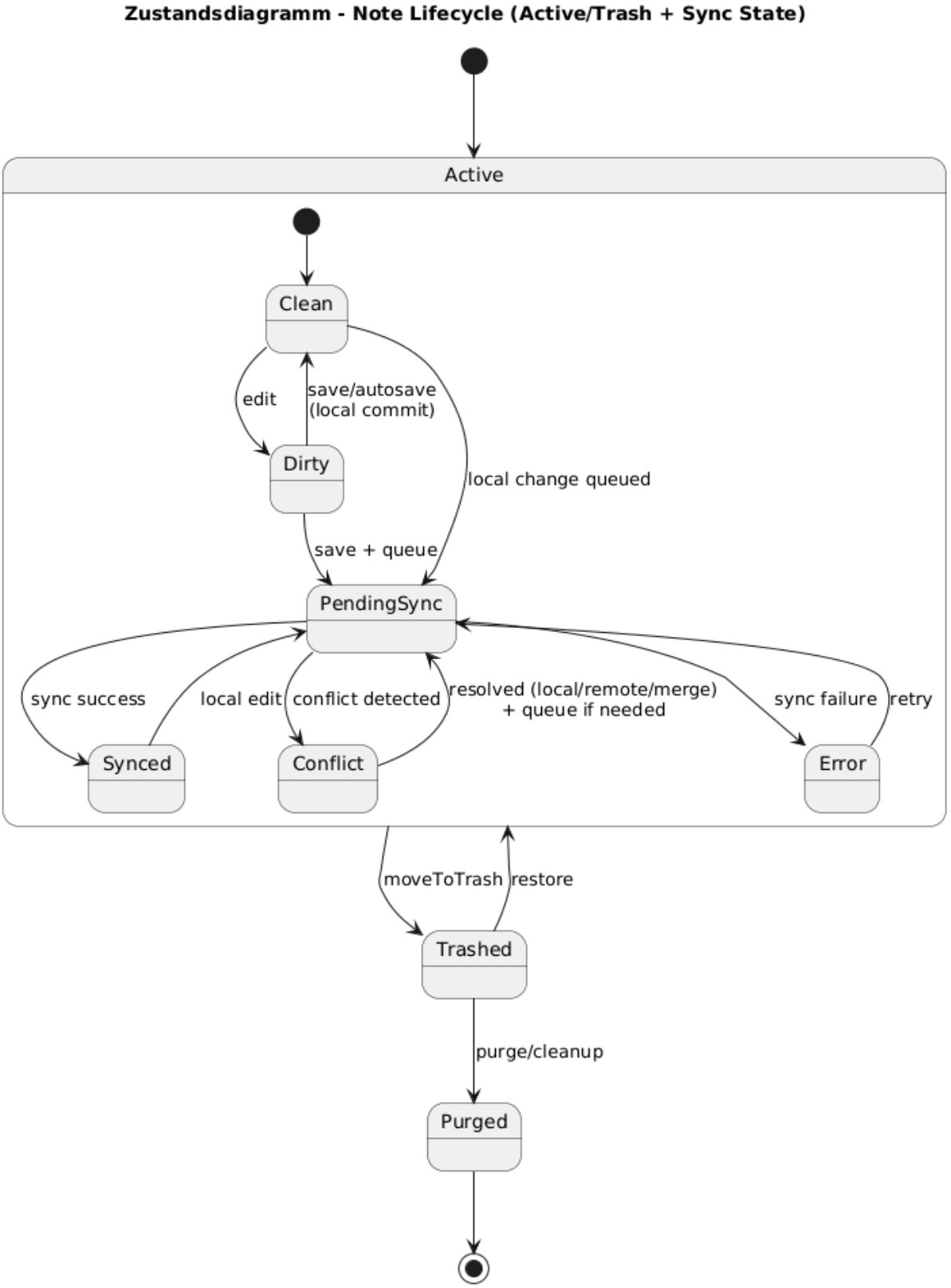
Beschreibung:
Dieses Diagramm stellt den Prozess dar, der ausgeführt wird, wenn eine Notiz synchronisiert wird. Wichtig ist der Loop über die Konflikterkennung und -Anpassung.



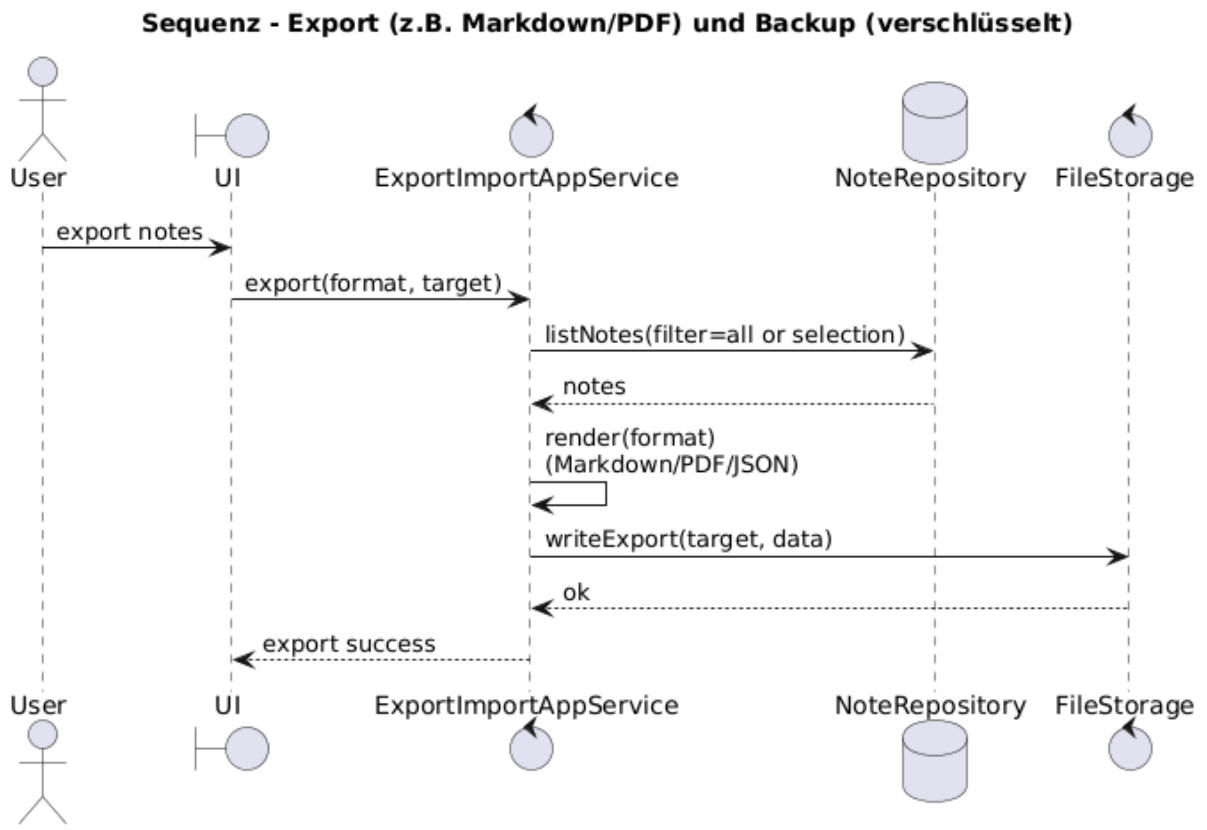
Beschreibung:
Wenn ein Konflikt erkannt wird, kann der User eine Strategie zum Auflösen des Konfliktes angeben, welches in diesem Diagramm näher beschrieben wird



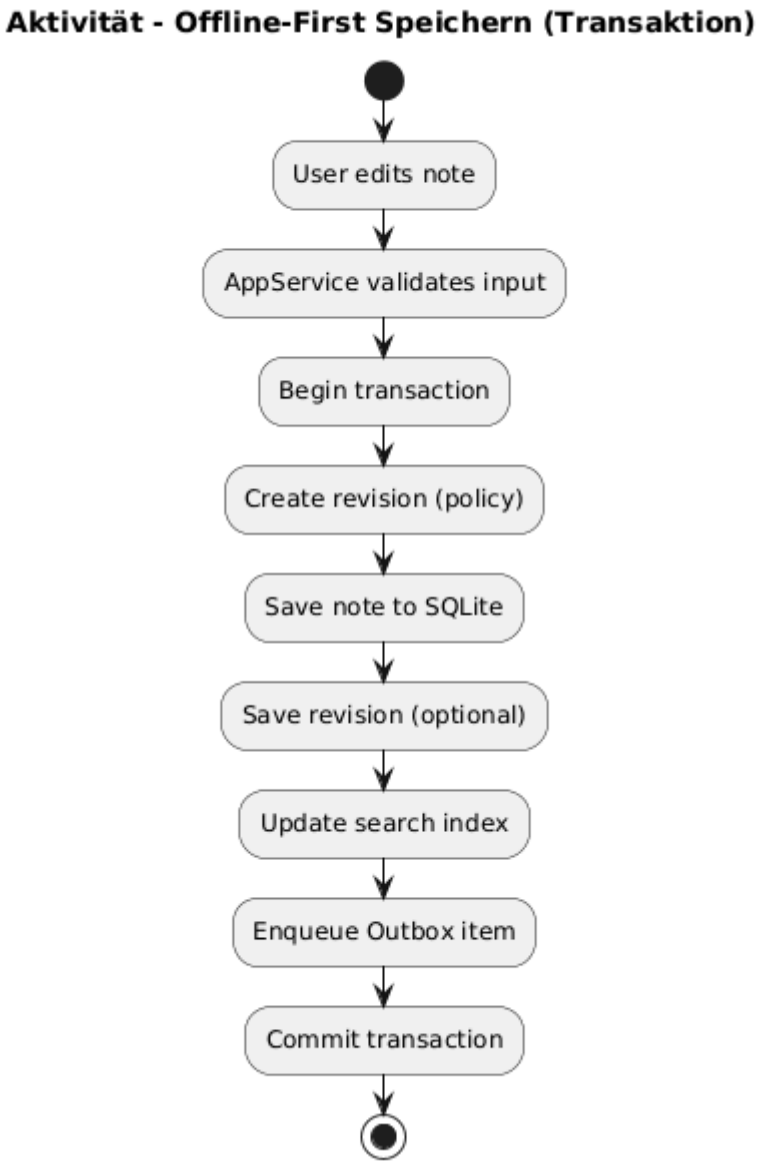
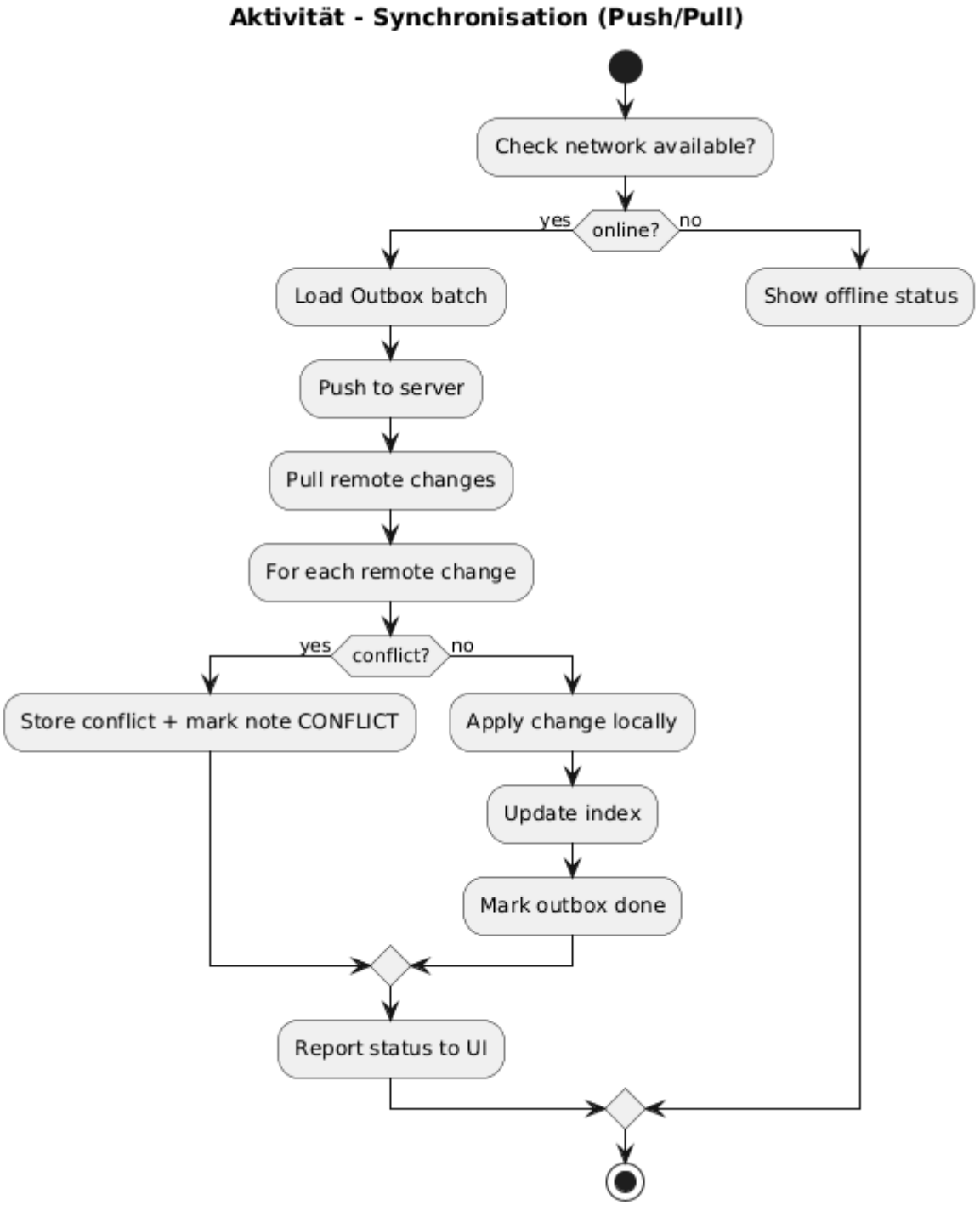
Beschreibung:
Dieses Diagramm stellt den Lifecycle Prozess einer Notiz dar.



Beschreibung:
Dieses Diagramm stellt den Prozess des Exports in das Speichersystem dar.



Beschreibung:
Dieses Diagramm stellt den Ablauf der Offline-Speicherung dar.



Beschreibung:
Dieses Diagramm stellt den Ablauf der Synchronisation dar.